1. Feature and Class level.
   Naive Bayes assumes that the **features are conditionally independent given the class label**.
   Mathematically:

$$P(X_1, X_2, \dots, X_n \mid Y) = \prod_{i=1}^{n} P(X_i \mid Y)$$

   where:

- X_i= features
- Y= class label

   So the statement becomes:
   The Naive Bayes classifier makes the assumption that the features are independent given the class labels.

2. The **Naive Bayes** classifier can produce a **non-linear decision boundary**, depending on the probability distributions assumed for the features.
   For example, in **Gaussian Naive Bayes**, the class-conditional probabilities are modeled using Gaussian distributions. When different classes have different variances, the resulting decision boundary can be **quadratic (non-linear)**.
3. Class Imbalance
   Explanation
   In **One-vs-Rest (OvR)** multi-class classification, a separate classifier is trained for each class:

- Class 1 vs. all other classes
- Class 2 vs. all other classes
- Class 3 vs. all other classes
- etc.

   This can create a **class imbalance problem**, because the positive class may contain far fewer samples than the combined negative classes.
   For example, with 5 classes having 100 samples each:

- Class 1 → 100 positive vs. **400 negative**
- Class 2 → 100 positive vs. **400 negative**

4. Answer: 9
   We have:

Y= Boolean → **2 classes**

X_1= categorical with **3 values**

X_2= continuous and **normally distributed**

For a Naive Bayes model:

1. Prior probability of Y

Since $Y$is Boolean, we need only **1 parameter**:

$$P(Y = 1)$$

(The probability of the other class is $1 - P(Y = 1)$.)

2. Conditional probabilities for X_1

For each of the 2 values of Y, X_1has 3 possible values.

A categorical variable with 3 values requires $3 - 1 = 2$independent parameters per class:

$$2 \times 2 = 4$$

3. Gaussian parameters for X_2

For each value of Y, a Gaussian distribution requires:

Mean μ

Variance σ^2

So:

$$2 \text{ parameters} \times 2 \text{ classes} = 4$$

Total 1+4+4=9

5. Answer: Increased

In **boosting** algorithms such as **AdaBoost**, data points that are **misclassified** by the current weak learner are given **higher weights** in the next iteration.

This forces the next learner to pay more attention to the difficult or incorrectly classified examples.

So, as training progresses:

Weights of misclassified points are increased

6. Option 2 and 3

- **Decreasing** m→ each tree sees fewer randomly selected features → trees become **less correlated** with each other.

- **Increasing** m→ each tree has more features to choose from → it generally makes **stronger individual trees**.

7. **Option D**
8. **None of them can be inferred.**